

TP N° 04

Base de données - migration

Configuration de la BD :

Voir le fichier de configuration .env :

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=homestead
DB_USERNAME=homestead
DB_PASSWORD=secret
```

RQ : vérifier l'existence de la base.

Les migrations

Une migration permet de créer et de mettre à jour un schéma de base de données.

Artisan

Testez la commande suivante :

```
php artisan
```

Voir la partie des migrations :

```
migrate
migrate:fresh      Drop all tables and re-run all migrations
migrate:install    Create the migration repository
migrate:refresh    Reset and re-run all migrations
migrate:reset      Rollback all database migrations
migrate:rollback   Rollback the last database migration
migrate:status     Show the status of each migration
```

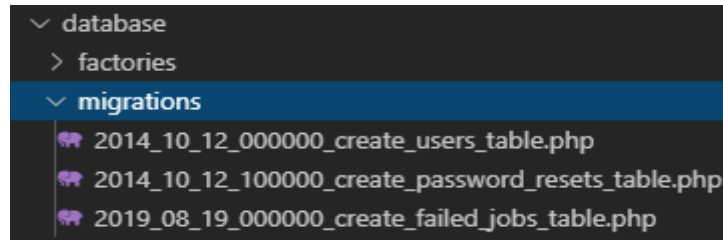
On dispose de 6 commandes :

- **fresh** : supprime toutes les tables et relance la migration
- **install** : crée et informe la table de référence des migrations
- **refresh** : réinitialise et relance les migrations

- **rollback** : annule la dernière migration
- **status** : donne des informations sur les migrations

Installation :

Au départ il y a déjà trois migrations présentes dans le dossier database/migrations :



- Table **users** : c'est une migration de base pour créer une table des utilisateurs,
- Table **password_resets** : c'est une migration liée à la précédente qui permet de gérer le renouvellement des mots de passe en toute sécurité,
- Table **failed_jobs** : une migration qui concerne les queues.

Lancer la commande : création de la table de migration

```
php artisan migrate:install
```

Lancer les migrations

```
php artisan migrate
```

Création d'une migration

```
php artisan make:migration create_Clients_table --create=clients
```

Visualiser le dossier **database/migrations/**

On dispose dans cette classe de deux fonctions :

- **up** : ici on a le code de création de la table et de ses colonnes
- **down** : ici on a le code de suppression de la table

Note : l'ordre des migrations prend une grande importance lorsqu'on a des clés étrangères !

TAF : Soit le schéma des tables suivant :

MATIERE (codemat, libelle, coef),

EPREUVE (numepreuve, datepreuve, lieu),

- 1- Créer les migrations des tables 'epreuves' et 'matieres',
- 2- Modifier les migrations afin d'avoir les schémas des tables adéquats.[1]
- 3- Générer les tables.
- 4- Pour les deux contrôleurs « MatController » et « EprController » :[2][3]
 - a. Modifier la fonction « index » de façon quelle sélectionne les données à partir de la BD (utilisez l'affichage réalisé au TP précédent).
 - b. Ajoutez la fonction «store» pour insérer des données dans la BD.

Liens

Migrations, [1] <https://laravel.com/docs/9.x/migrations>

NOTE : Avec Laravel l'interaction avec les bases de données peut se faire avec trois méthodes :

- ❖ SQL brut, [2] <https://laravel.com/docs/9.x/database>
- ❖ Le générateur de requête fluide, [3] <https://laravel.com/docs/9.x/queries>
- ❖ L'ORM Eloquent : voir TP suivant.

→Tester les deux premières méthodes pour votre TP.

→Soit **tinker** : est un outil en ligne de commande permettant de tester rapidement du code dans son contexte.

Lancer Tinker

```
php artisan tinker
```

Ensuite, utilisez les commandes que vous souhaitez depuis le terminal et tinker affichera le résultat.

Interface de Tinker

```
>>>$users = DB::select('select * from users');  
>>> exit //pour quitter
```